

claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_d4359b2a-456\agent-a53c9336e651983a1.jsonl`  
- SHA-256: `c6443c72507687043026a49251e0047b7e8da73090e7823bc824a81bd03972ea`  
- Source modified: `2026-06-06T15:23:31+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_d4359b2a-456`  
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

Transcript

1. user (2026-06-06T15:20:37.520Z)

You are mapping ONE subsystem of the "badminton-highlight-indexer" Python repo (cwd is the repo root) for a dense CODE MAP that a future AI engineer will read to ramp up FAST on the codebase without crawling files.

Subsystem: Eval & calibration

Files: backend/eval/metrics.py, backend/eval/annotations.py, backend/eval/calibration.py, backend/eval/calibrate_wasb.py, backend/eval/export_wasb_segments.py, backend/eval/batch.py, backend/eval/harness.py, backend/eval/training.py, backend/eval/classifier.py, backend/eval/regression.py

READ each listed file (use Read; use Grep/Glob to resolve any directory globs to actual files). Then return, per file:

- path (exact, repo-relative, forward slashes)
- role: one terse line ? what the file is for
- symbols: the IMPORTANT classes / functions / dataclasses / module-level constants & registries. For each: name, kind (class|func|dataclass|const|registry), a terse signature (params + return where useful), and a one-line role. SKIP trivial private helpers and boilerplate.
- contracts: exact data shapes this file defines or consumes (dict keys, CSV column orders, tuple layouts, dataclass fields, return types). Be precise ? these are the cross-module interfaces.
- edges: calls/imports to OTHER files that matter for dataflow, each as "thisSymbol -> otherFile.thing (why)".
- gotchas: non-obvious constraints, footguns, platform/WSL/GPU/threadpool specifics, dead paths, defaults that bite.

Rules: be terse and ACCURATE. Every symbol and path MUST actually exist in the file ? do NOT invent or guess. If something is notable but you're unsure, omit it. Return via the structured schema.

2. assistant (2026-06-06T15:20:40.106Z)

I'll map the Eval & calibration subsystem. Let me start by reading all the listed files.

3. user (2026-06-06T15:20:40.996Z)

```
1      """Pure temporal interval-matching metrics for rally evaluation.  
2  
3      No I/O, no DB, no global state ? every function here is deterministic and unit  
4      tested. An "interval" is a `(start, end)` tuple of floats in seconds with  
5      start < end. We score a list of *predicted* rally intervals against a list of  
6      *ground-truth* intervals via temporal IoU and one-to-one greedy matching.  
7      """  
8
```

```

9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13
14
15    def interval_iou(a: Interval, b: Interval) -> float:
16        """Temporal intersection-over-union of two intervals.
17
18        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19        overlap / union, both measured as durations on the timeline.
20        """
21        a_start, a_end = a
22        b_start, b_end = b
23        if a_end <= a_start or b_end <= b_start:
24            return 0.0
25        inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26        if inter <= 0.0:
27            return 0.0
28        union = (a_end - a_start) + (b_end - b_start) - inter
29        return inter / union if union > 0 else 0.0
30
31
32    @dataclass
33    class Match:
34        """One matched predicted?ground-truth pair."""
35        pred_index: int
36        gt_index: int
37        iou: float
38
39
40    @dataclass
41    class MatchResult:
42        """Outcome of matching predictions to ground truth at a given IoU threshold."""
43        matches: List[Match] = field(default_factory=list)
44        unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45        unmatched_gt_indices: List[int] = field(default_factory=list)   # false negatives
46(misses)
47
48    def match_intervals(preds: List[Interval], gts: List[Interval],
49                        iou_threshold: float = 0.5) -> MatchResult:
50        """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52        Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53        are claimed highest-IoU first, and each prediction/GT can be used once. This
54        keeps a single prediction from being credited for two ground-truth rallies
55        (and vice versa).
56        """
57        candidates = []
58        for pi, p in enumerate(preds):
59            for gi, g in enumerate(gts):
60                iou = interval_iou(p, g)
61                if iou >= iou_threshold:
62                    candidates.append((iou, pi, gi))
63        # Highest IoU first; ties broken by indices for determinism.
64        candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
65
66        used_pred, used_gt = set(), set()
67        matches: List[Match] = []
68        for iou, pi, gi in candidates:
69            if pi in used_pred or gi in used_gt:
70                continue
71            used_pred.add(pi)
72            used_gt.add(gi)

```

```

73         matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
74
75     unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
76     unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
77     return MatchResult(matches=matches,
78                        unmatched_pred_indices=unmatched_pred,
79                        unmatched_gt_indices=unmatched_gt)
80
81
82     @dataclass
83     class Score:
84         """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
85         iou_threshold: float
86         num_pred: int
87         num_gt: int
88         true_positives: int
89         false_positives: int
90         false_negatives: int
91         precision: float
92         recall: float
93         f1: float
94         mean_iou: float          # over matched pairs (0.0 if none)
95         mean_start_error: float   # mean |pred_start - gt_start| over matches, seconds
96         mean_end_error: float     # mean |pred_end - gt_end| over matches, seconds
97
98         def as_dict(self) -> dict:
99             return self.__dict__.copy()
100
101
102     def score(preds: List[Interval], gts: List[Interval],
103              iou_threshold: float = 0.5,
104              match_result: Optional[MatchResult] = None) -> Score:
105         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
106
107         Pass a precomputed `match_result` to avoid re-matching (e.g. when also
108         extracting the failure lists); otherwise matching is done here.
109         """
110         mr = match_result if match_result is not None else match_intervals(preds, gts,
111 iou_threshold)
112
113         tp = len(mr.matches)
114         fp = len(mr.unmatched_pred_indices)
115         fn = len(mr.unmatched_gt_indices)
116
117         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
118         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
119         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
120 else 0.0
121
122         if mr.matches:
123             mean_iou = sum(m.iou for m in mr.matches) / tp
124             mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
125 mr.matches) / tp
126             mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
127 mr.matches) / tp
128         else:
129             mean_iou = mean_start_err = mean_end_err = 0.0
130
131         return Score(
132             iou_threshold=iou_threshold,
133             num_pred=len(preds), num_gt=len(gts),
134             true_positives=tp, false_positives=fp, false_negatives=fn,
135             precision=precision, recall=recall, f1=f1,
136             mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
137 )

```

```

134
135
136 def pr_curve(preds: List[Interval], gts: List[Interval],
137             thresholds: Optional[List[float]] = None) -> List[Score]:
138     """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
139
140     Lets you see strict-vs-lenient behaviour rather than a single operating point.
141     """
142     if thresholds is None:
143         thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
144     return [score(preds, gts, iou_threshold=t) for t in thresholds]
145

```

4. user (2026-06-06T15:20:41.909Z)

```

1     """Load hand-annotated ground-truth rally timestamps from CSV.
2
3     Annotation file format (one file per video):
4
5         start,end
6         00:01:12,00:01:39
7         102.5,131.0
8
9     - A header row `start,end` is optional (auto-detected and skipped).
10    - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11      `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12    - Blank lines and `#` comment lines are ignored.
13    """
14
15    import csv
16    import logging
17    from typing import List, Tuple
18
19    logger = logging.getLogger(__name__)
20
21    Interval = Tuple[float, float]
22
23
24    def parse_timestamp(value: str) -> float:
25        """Parse a timestamp string to float seconds.
26
27        Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28        Raises ValueError on anything unparseable so bad annotations fail loudly.
29        """
30        value = value.strip()
31        if not value:
32            raise ValueError("empty timestamp")
33        if ":" in value:
34            parts = value.split(":")
35            if len(parts) > 3:
36                raise ValueError(f"too many ':' segments in timestamp '{value}'")
37            total = 0.0
38            for part in parts: # most-significant first (H, M, S)
39                total = total * 60 + float(part)
40            return total
41        return float(value)
42
43
44    def load_annotations(csv_path: str) -> List[Interval]:
45        """Load and validate ground-truth rally intervals from a CSV file.
46
47        Returns intervals sorted by start time. Raises ValueError on malformed rows
48        (bad timestamp, start >= end) so labeling mistakes surface immediately rather
49        than silently skewing scores.
50        """

```

```

51     intervals: List[Interval] = []
52     with open(csv_path, newline="") as f:
53         reader = csv.reader(f)
54         for lineno, row in enumerate(reader, start=1):
55             if not row:
56                 continue
57             first = row[0].strip()
58             if not first or first.startswith("#"):
59                 continue
60             # Skip an optional header row.
61             if first.lower() in ("start", "start_time"):
62                 continue
63             if len(row) < 2:
64                 raise ValueError(f"{csv_path}:{lineno}: expected 'start,end', got
{row!r}")
65             try:
66                 start = parse_timestamp(row[0])
67                 end = parse_timestamp(row[1])
68             except ValueError as e:
69                 raise ValueError(f"{csv_path}:{lineno}: {e}") from e
70             if start >= end:
71                 raise ValueError(f"{csv_path}:{lineno}: start ({start}) >= end ({end}")
72             intervals.append((start, end))
73
74     intervals.sort(key=lambda iv: iv[0])
75     logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
76     return intervals
77
78
79 def write_scaffold(csv_path: str) -> None:
80     """Write an empty annotation file with just the header, for hand-filling."""
81     with open(csv_path, "w", newline="") as f:
82         writer = csv.writer(f)
83         writer.writerow(["start", "end"])
84

```

5. user (2026-06-06T15:20:42.367Z)

```

1     """Quantify rally-segmentation improvement ? and guard against overfitting.
2
3     This is the measurement layer for per-video / per-corpus calibration. It answers
4     two questions honestly:
5
6     1. """Did precision improve, by how much?""" ? `improvement_report` scores a
7         baseline config vs a tuned config on the same ground truth and reports the
8         before/after metrics and the % delta.
9
10    2. """Is that gain real or overfit?""" ? calibration must be measured on data the
11        tuning never saw, otherwise you're just memorising. Two protocols:
12        - `cross_validate` ? k-fold over one video's rallies: pick the config on the
13            train folds, score on the held-out fold. The generalization_gap
14            (train ? test) is the overfit alarm.
15        - `leave_one_video_out` ? the protocol that matters as more videos arrive:
16            pick the config on all other videos, score on the held-out video. Reports
17            per-video held-out scores + mean±std. This is the number to trust.
18
19    Everything is segmenter-agnostic: callers pass a `predict_fn(config) ->
20    List[Interval]` (e.g. WASB trajectory ? windows at those thresholds), so the same
21    harness works for WASB, yolo_hybrid, or anything else. Pure + unit-tested; no I/O.
22    """
23    from __future__ import annotations
24
25    import statistics as _st
26    from typing import Callable, Dict, List, Sequence, Tuple, Any
27

```

```

28     from backend.eval.metrics import score, Score, Interval
29
30     Config = Any                                # opaque to this module (e.g. a thresholds
dict)
31     PredictFn = Callable[[Config], List[Interval]]
32
33
34     def pct_improvement(before: float, after: float) -> float:
35         """Percentage change from `before` to `after` (e.g. 0.6 -> 0.75 == +25.0)."""
36         if before <= 0:
37             return float("inf") if after > 0 else 0.0
38         return (after - before) / before * 100.0
39
40
41     def evaluate(preds: List[Interval], gts: List[Interval], iou_threshold: float = 0.5) ->
Score:
42         """Precision / recall / F1 / mean-IoU of predicted vs ground-truth intervals."""
43         return score(preds, gts, iou_threshold)
44
45
46     def kfold_indices(n: int, k: int) -> List[Tuple[List[int], List[int]]]:
47         """Deterministic k-fold split of indices 0..n-1 into (train_idx, test_idx)."""
48         if n <= 0:
49             return []
50         k = max(2, min(k, n))                # need >=2 folds; cap at n
51         folds = [list(range(i, n, k)) for i in range(k)]    # round-robin ? balanced,
deterministic
52         splits = []
53         for f in range(k):
54             test = folds[f]
55             train = [i for i in range(n) if i not in set(test)]
56             if test and train:
57                 splits.append((train, test))
58         return splits
59
60
61     def select_best(predict_fn: PredictFn, configs: Sequence[Config], gts: List[Interval],
62                     metric: str = "f1", iou_threshold: float = 0.5) -> Tuple[Config, float]:
63         """Pick the config whose predictions maximise `metric` against `gts`."""
64         best_cfg, best_val = None, -1.0
65         for cfg in configs:
66             val = getattr(score(predict_fn(cfg), gts, iou_threshold), metric)
67             if val > best_val:
68                 best_cfg, best_val = cfg, val
69         return best_cfg, best_val
70
71
72     def cross_validate(predict_fn: PredictFn, configs: Sequence[Config], gts:
List[Interval],
73                       k: int = 5, metric: str = "recall", iou_threshold: float = 0.5) ->
Dict[str, Any]:
74         """k-fold threshold calibration on ONE video's rallies (within-video overfit guard).
75
76         Per fold: choose the best config on the train rallies, score it on the held-out
77         rallies. A large `generalization_gap` (train >> test) means the thresholds are
78         overfitting that video's quirks.
79
80         Default metric is **recall** on purpose: predictions are global (whole-video
81         windows), so scoring them against a *fold subset* of GT would unfairly count the
82         other rallies' windows as false positives ? precision/F1 are not well-defined
83         per-fold. Recall ("did the tuned thresholds still detect the held-out rallies?")
84         is the honest within-video generalization signal. For precision/F1, use
85         `leave_one_video_out` (each video scored whole) or `improvement_report` on a
86         held-out video.
87         """

```

```

88     splits = kfold_indices(len(gts), k)
89     if not splits:
90         return {"error": "not enough rallies for cross-validation", "n": len(gts)}
91     train_scores, test_scores, picked = [], [], []
92     for train_idx, test_idx in splits:
93         gts_tr = [gts[i] for i in train_idx]
94         gts_te = [gts[i] for i in test_idx]
95         cfg, tr = select_best(predict_fn, configs, gts_tr, metric, iou_threshold)
96         te = getattr(score(predict_fn(cfg), gts_te, iou_threshold), metric)
97         train_scores.append(tr)
98         test_scores.append(te)
99         picked.append(cfg)
100     mean_test = _st.mean(test_scores)
101     mean_train = _st.mean(train_scores)
102     return {
103         "metric": metric,
104         "folds": len(splits),
105         "mean_test": mean_test,
106         "std_test": _st.pstdev(test_scores) if len(test_scores) > 1 else 0.0,
107         "mean_train": mean_train,
108         "generalization_gap": mean_train - mean_test,    # >=0.1 = likely overfitting
109         "picked_configs": picked,
110     }
111
112
113 def leave_one_video_out(videos: List[Dict[str, Any]], configs: Sequence[Config],
114                        metric: str = "f1", iou_threshold: float = 0.5) -> Dict[str,
115 Any]:
116     """Leave-one-video-out CV ? the honest metric as the corpus grows.
117
118     `videos`: list of {"name": str, "predict_fn": PredictFn, "gts": [Interval]}.
119     For each held-out video: pick the config that's best *pooled across the other
120     videos*, then score it on the held-out one. The mean held-out score is the
121     "precision on unseen videos" number; per-video spread shows stability.
122     """
123     if len(videos) < 2:
124         return {"error": "need >=2 videos for leave-one-video-out", "n": len(videos)}
125     per_video = []
126     for i, held in enumerate(videos):
127         others = [v for j, v in enumerate(videos) if j != i]
128
129         def pooled_score(cfg: Config) -> float:
130             vals = [getattr(score(v["predict_fn"](cfg), v["gts"], iou_threshold),
131 metric)
132                        for v in others]
133             return _st.mean(vals)
134
135         best_cfg, best_train = max(((c, pooled_score(c)) for c in configs), key=lambda
136 t: t[1])
137         held_score = getattr(score(held["predict_fn"](best_cfg), held["gts"],
138 iou_threshold), metric)
139         per_video.append({"video": held.get("name", f"video_{i}"),
140 "held_out": held_score, "train_pooled": best_train, "config":
141 best_cfg})
142     held_scores = [p["held_out"] for p in per_video]
143     train_scores = [p["train_pooled"] for p in per_video]
144     return {
145         "metric": metric,
146         "mean_held_out": _st.mean(held_scores),
147         "std_held_out": _st.pstdev(held_scores) if len(held_scores) > 1 else 0.0,
148         "generalization_gap": _st.mean(train_scores) - _st.mean(held_scores),
149         "per_video": per_video,
150     }

```

```

148     def improvement_report(predict_fn: PredictFn, baseline_config: Config, tuned_config:
Config,
149                             gts: List[Interval], iou_threshold: float = 0.5) -> Dict[str,
Any]:
150         """The "+X%" precision" headline: baseline vs tuned config on the same GT.
151
152         NB: for an *honest* headline, pass GT the tuning never saw (a held-out video),
153         or read the cross-validated `mean_test` from `cross_validate` instead.
154         """
155         b = score(predict_fn(baseline_config), gts, iou_threshold)
156         t = score(predict_fn(tuned_config), gts, iou_threshold)
157         return {
158             "baseline": {"precision": b.precision, "recall": b.recall, "f1": b.f1,
"mean_iou": b.mean_iou},
159             "tuned": {"precision": t.precision, "recall": t.recall, "f1": t.f1, "mean_iou":
t.mean_iou},
160             "precision_delta_pts": t.precision - b.precision,
161             "precision_delta_pct": pct_improvement(b.precision, t.precision),
162             "f1_delta_pct": pct_improvement(b.f1, t.f1),
163         }
164

```

6. user (2026-06-06T15:20:42.741Z)

```

1     """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3     Wires the already-tested pieces together:
4     WASB trajectory CSV (Frame,Visibility,X,Y)
5     -> TrackNetRunner.trajectory_to_action_windows(cfg) [predict_fn]
6     -> backend.eval.calibration (improvement_report / cross_validate)
7
8     Run (after a full-video WASB pass exists):
9     python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10         --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12     Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13     same GT is an **optimistic** upper bound. The trustworthy number is the
14     cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15     `leave_one_video_out` for precision/F1 generalization).
16     """
17     from __future__ import annotations
18
19     import csv
20     import argparse
21     from typing import List, Tuple, Callable
22
23     from backend.detectors.tracknet_runner import TrackNetRunner
24     from backend.eval.calibration import (
25         select_best, improvement_report, cross_validate, evaluate,
26     )
27
28     Interval = Tuple[float, float]
29     Config = Tuple[float, float, float] # (velocity_thresh, merge_gap,
min_window_duration)
30
31     BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig / trajectory_to_action_windows
defaults
32
33
34     def load_golden_gts(csv_path: str) -> List[Interval]:
35         """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
36         gts: List[Interval] = []
37         with open(csv_path, newline="", encoding="utf-8") as f:
38             reader = csv.DictReader(f)

```

```

39         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
40     for row in reader:
41         row = {(k or "").strip().lower(): v for k, v in row.items()}
42         try:
43             s = float((row.get("start_time") or "").strip())
44             e = float((row.get("end_time") or "").strip())
45         except (TypeError, ValueError):
46             continue
47         if e > s:
48             gts.append((s, e))
49     return sorted(gts)
50
51
52     def make_predict_fn(trajecory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
53         """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
54         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
55
56         def predict_fn(cfg: Config) -> List[Interval]:
57             vt, mg, mwd = cfg
58             wins = TrackNetRunner.trajectory_to_action_windows(
59                 points, fps=fps, frame_width=frame_width,
60                 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
61             )
62             return [(w["start"], w["end"]) for w in wins]
63
64         return predict_fn
65
66
67     def default_grid() -> List[Config]:
68         return [(vt, mg, mwd)
69                 for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
70                 for mg in (1.0, 2.0, 3.0)
71                 for mwd in (1.0, 2.0, 3.0)]
72
73
74     def run(trajecory_csv: str, labels_csv: str, fps: float, frame_width: float,
75            iou: float = 0.5) -> dict:
76         gts = load_golden_gts(labels_csv)
77         if not gts:
78             raise SystemExit(f"no usable golden rallies in {labels_csv}")
79         predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
80         grid = default_grid()
81
82         base = evaluate(predict_fn(BASELINE), gts, iou)
83         best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
84         fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou) # OPTIMISTIC
(fit on full GT)
85         cv = cross_validate(predict_fn, grid, gts, k=5) # HONEST held-
out recall
86
87         print("=== WASB rally-segmentation calibration ===")
88         print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")
89         print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
90               f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
91         print(f"tuned cfg {best_cfg}: f1={best_f1:.3f}")
92         print(f" precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
93               f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
94         print(f" f1 {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
95               f"({fit['f1_delta_pct']:+.1f}%)")
96         print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall ")

```

```

97         f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
98         f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
99     print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
100     return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
101
102
103     def main():
104         ap = argparse.ArgumentParser()
105         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
106         ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
107         ap.add_argument("--fps", type=float, default=59.94)
108         ap.add_argument("--frame-width", type=float, default=1920.0)
109         ap.add_argument("--iou", type=float, default=0.5)
110         args = ap.parse_args()
111         run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
112
113
114     if __name__ == "__main__":
115         main()
116

```

7. assistant (2026-06-06T15:20:44.986Z)

Let me read the remaining files.

8. user (2026-06-06T15:20:46.115Z)

```

1     """Export tuned WASB rally segments for manual eyeball verification.
2
3     After calibration picks a tuned threshold config, this turns a cached WASB
4     trajectory into the actual rally [start,end] segments that config produces, so the
5     user can answer "are the rallies actually better?" ? by scrubbing the video to each
6     segment, by diffing tuned-vs-golden quantitatively, or by cutting clips.
7
8     Reuses (no new windowing/metrics/cutting logic):
9     - calibrate_wasb.make_predict_fn -> trajectory -> segments for a given config
10    - eval.metrics.match_intervals/score -> tuned-vs-golden IoU comparison
11    - tools.rally_slicer.slice_rallies -> optional clip cutting (the exported CSV is
12      written in the golden/slicer schema, so it feeds straight back in)
13
14    Run (uses the tuned config from the first calibration by default):
15    python -m backend.eval.export_wasb_segments --trajectory
~/clips/sample_long_traj.csv \
16        --labels "Annotation Setup/golden_labels_badminton.csv" --fps 59.94 --frame-
width 1920
17        # add --video <mp4> --slice to also cut one padded clip per tuned rally
18    """
19    from __future__ import annotations
20
21    import csv
22    import os
23    import os.path as osp
24    import argparse
25    from typing import List, Tuple, Dict, Optional
26
27    from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
28    from backend.eval.metrics import match_intervals, score
29
30    Interval = Tuple[float, float]
31    Config = Tuple[float, float, float]
32
33    # The tuned config from the first per-video calibration (velocity, merge_gap, min_dur);

```

```

34     # F1 0.588 (baseline) -> 0.746. Override on the CLI to export a different operating
point.
35     TUNED: Config = (0.05, 1.0, 1.0)
36
37     SEG_FIELDS = ["rally_number", "start_time", "end_time", "duration", "start_mmss",
"end_mmss"]
38     COMP_FIELDS = ["type", "ref", "start_time", "end_time", "mmss", "status",
39                     "iou", "pred_start", "pred_end", "start_err_s", "end_err_s"]
40
41
42     def seconds_to_mmss(t: float) -> str:
43         """Human-scrubbable timestamp, e.g. 106.031 -> '1:46.031'."""
44         t = max(0.0, float(t))
45         m = int(t // 60)
46         return f"{m}:{t - m * 60:06.3f}"
47
48
49     def segments_to_rows(segments: List[Interval]) -> List[Dict[str, str]]:
50         """Number segments 1..N in time order; emit the golden/slicer CSV schema."""
51         rows = []
52         for i, (s, e) in enumerate(segments, start=1):
53             rows.append({
54                 "rally_number": str(i),
55                 "start_time": f"{s:.3f}",
56                 "end_time": f"{e:.3f}",
57                 "duration": f"{e - s:.3f}",
58                 "start_mmss": seconds_to_mmss(s),
59                 "end_mmss": seconds_to_mmss(e),
60             })
61         return rows
62
63
64     def _ensure_parent(path: str) -> None:
65         os.makedirs(osp.dirname(osp.abspath(path)) or ".", exist_ok=True)
66
67
68     def write_segments_csv(path: str, segments: List[Interval]) -> str:
69         """Write segments in the golden schema (rally_number/start_time/end_time + mm:ss).
70
71         The schema is what tools.rally_slicer.load_rally_labels consumes, so the file
72         doubles as input to clip cutting.
73         """
74         _ensure_parent(path)
75         with open(path, "w", newline="", encoding="utf-8") as f:
76             w = csv.DictWriter(f, fieldnames=SEG_FIELDS)
77             w.writeheader()
78             w.writerows(segments_to_rows(segments))
79         return path
80
81
82     def build_comparison(preds: List[Interval], gts: List[Interval],
83                         iou_threshold: float = 0.5) -> dict:
84         """Golden-centric comparison: per golden rally, did a tuned segment match (IoU),
85         plus the boundary errors and any extra (false-positive) tuned segments."""
86         mr = match_intervals(preds, gts, iou_threshold)
87         sc = score(preds, gts, iou_threshold, match_result=mr)
88         by_gt = {m.gt_index: m for m in mr.matches}
89
90         rows: List[Dict[str, str]] = []
91         for gi, (gs, ge) in enumerate(gts, start=0):
92             m = by_gt.get(gi)
93             if m is not None:
94                 ps, pe = preds[m.pred_index]
95                 rows.append({
96                     "type": "golden", "ref": str(gi + 1),

```

```

97         "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
98         "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
99         "status": "matched", "iou": f"{m.iou:.3f}",
100        "pred_start": f"{ps:.3f}", "pred_end": f"{pe:.3f}",
101        "start_err_s": f"{ps - gs:+.3f}", "end_err_s": f"{pe - ge:+.3f}",
102    })
103    else:
104        rows.append({
105            "type": "golden", "ref": str(gi + 1),
106            "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
107            "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
108            "status": "MISS", "iou": "0.000",
109            "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
110        })
111    for pi in mr.unmatched_pred_indices: # extra tuned segments = false
positives
112        ps, pe = preds[pi]
113        rows.append({
114            "type": "pred_extra", "ref": str(pi + 1),
115            "start_time": f"{ps:.3f}", "end_time": f"{pe:.3f}",
116            "mmss": f"{seconds_to_mmss(ps)}-{seconds_to_mmss(pe)}",
117            "status": "false_positive", "iou": "",
118            "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
119        })
120    return {"score": sc, "rows": rows}
121
122
123    def write_comparison_csv(path: str, comparison: dict) -> str:
124        _ensure_parent(path)
125        with open(path, "w", newline="", encoding="utf-8") as f:
126            w = csv.DictWriter(f, fieldnames=COMP_FIELDS)
127            w.writeheader()
128            w.writerows(comparison["rows"])
129        return path
130
131
132    def _with_suffix(path: str, suffix: str) -> str:
133        root, ext = osp.splitext(path)
134        return root + suffix + ext
135
136
137    def run(trajecory_csv: str, fps: float, frame_width: float, cfg: Config = TUNED,
138            out_csv: str = "output/wasb_tuned_segments.csv", labels_csv: Optional[str] =
None,
139            iou: float = 0.5, baseline: bool = False,
140            comparison_out: Optional[str] = None) -> dict:
141        predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
142        tuned_segs = predict_fn(cfg)
143        write_segments_csv(out_csv, tuned_segs)
144        total = sum(e - s for s, e in tuned_segs)
145        print("=== WASB tuned-segment export ===")
146        print(f"tuned cfg {cfg}: {len(tuned_segs)} rally segments, {total:.1f}s of play ->
{out_csv}")
147        result: dict = {"tuned_cfg": cfg, "num_tuned": len(tuned_segs), "out_csv": out_csv}
148
149        if baseline:
150            base_segs = predict_fn(BASELINE)
151            base_out = _with_suffix(out_csv, "_baseline")
152            write_segments_csv(base_out, base_segs)
153            print(f"baseline cfg {BASELINE}: {len(base_segs)} segments -> {base_out}")
154            result["num_baseline"] = len(base_segs)
155            result["baseline_csv"] = base_out
156
157        if labels_csv:
158            gts = load_golden_gts(labels_csv)

```

```

159         if not gts:
160             raise SystemExit(f"no usable golden rallies in {labels_csv}")
161         comp = build_comparison(tuned_segs, gts, iou)
162         sc = comp["score"]
163         comp_out = comparison_out or _with_suffix(out_csv, "_vs_golden")
164         write_comparison_csv(comp_out, comp)
165         print(f"\n--- tuned vs golden ({len(gts)} rallies, IoU>={iou}) ---")
166         print(f"matched (TP) {sc.true_positives} | misses (FN) {sc.false_negatives} | "
167               f"extra (FP) {sc.false_positives}")
168         print(f"precision {sc.precision:.3f} | recall {sc.recall:.3f} | f1 {sc.f1:.3f} | "
169               f"meanIoU {sc.mean_iou:.3f}")
170         print(f"mean boundary error: start {sc.mean_start_error:.2f}s | end "
171               f"{sc.mean_end_error:.2f}s")
172         result["comparison"] = sc.as_dict()
173         result["comparison_csv"] = comp_out
174     return result
175
176
177     def _maybe_slice(video: str, segments_csv: str, clips_dir: str) -> None:
178         """Cut one padded clip per exported rally (reuses the rally slicer)."""
179         from backend.tools.rally_slicer import slice_rallies
180         written = slice_rallies(video, segments_csv, ["all"], clips_dir)
181         print(f"\ncut {len(written)} clip(s) -> {clips_dir}")
182
183
184     def main() -> None:
185         ap = argparse.ArgumentParser(description="Export tuned WASB rally segments for "
186               "eyeballing.")
187         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV "
188               "(Frame,Visibility,X,Y)")
189         ap.add_argument("--fps", type=float, default=59.94)
190         ap.add_argument("--frame-width", type=float, default=1920.0)
191         ap.add_argument("--velocity-thresh", type=float, default=TUNED[0])
192         ap.add_argument("--merge-gap", type=float, default=TUNED[1])
193         ap.add_argument("--min-duration", type=float, default=TUNED[2])
194         ap.add_argument("--labels", default=None, help="golden labels CSV -> tuned-vs-golden "
195               "comparison")
196         ap.add_argument("--iou", type=float, default=0.5)
197         ap.add_argument("--out", default="output/wasb_tuned_segments.csv")
198         ap.add_argument("--comparison-out", default=None)
199         ap.add_argument("--baseline", action="store_true", help="also export baseline-config "
200               "segments")
201         ap.add_argument("--video", default=None, help="source video (with --slice) to cut "
202               "clips from")
203         ap.add_argument("--slice", action="store_true", help="cut one padded clip per tuned "
204               "rally")
205         ap.add_argument("--clips-dir", default="output/wasb_tuned_clips")
206         args = ap.parse_args()
207
208         cfg = (args.velocity_thresh, args.merge_gap, args.min_duration)
209         res = run(args.trajectory, args.fps, args.frame_width, cfg=cfg, out_csv=args.out,
210               labels_csv=args.labels, iou=args.iou, baseline=args.baseline,
211               comparison_out=args.comparison_out)
212         if args.slice:
213             if not args.video:
214                 raise SystemExit("--slice requires --video")
215             _maybe_slice(args.video, res["out_csv"], args.clips_dir)
216
217
218     if __name__ == "__main__":
219         main()
220

```

9. user (2026-06-06T15:20:46.427Z)

```
1      """Batch evaluation across a collector export manifest.
2
3      Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4      sports-data-collector's `curate export`, (optionally) process each video through
5      a segmenter so detections land in the indexer DB, then score every video's
6      detected rallies against its ground-truth CSV and aggregate the results.
7
8      This module holds the *pure* pieces (path resolution, stage selection, metric
9      aggregation) so they can be unit-tested without touching video files, the GPU,
10     or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
11     """
12
13     import os
14     from typing import Dict, List, Optional
15
16
17     # Segmenters that only ever populate the `final` play_segments table (no raw
18     # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
19     _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
20
21
22     def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23         """Resolve a manifest `local_path` to an absolute path.
24
25         The collector stores paths like ``./data/videos\\shuttle1.mp4`` relative
26         to its own repo root and with Windows separators. Normalise separators, strip
27         a leading ``./``, and resolve relative paths against `videos_root` (the
28         collector root). Returns None for a missing/empty path.
29         """
30         if not local_path:
31             return None
32         p = local_path.replace("\\", "/")
33         if p.startswith("./"):
34             p = p[2:]
35         if os.path.isabs(p):
36             return os.path.normpath(p)
37         return os.path.normpath(os.path.join(videos_root, p))
38
39
40     def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
41         """Pick which prediction stages to score.
42
43         ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
44         but only ``final`` for segmenters that don't emit candidates. An explicit
45         request ('candidates' | 'final' | 'both') overrides the heuristic.
46         """
47         if requested == "both":
48             return ["candidates", "final"]
49         if requested in ("candidates", "final"):
50             return [requested]
51         # auto
52         if segmenter in _FINAL_ONLY_SEGMENTERS:
53             return ["final"]
54         return ["candidates", "final"]
55
56
57     def _safe_div(num: float, den: float) -> float:
58         return num / den if den else 0.0
59
60
61     def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
62         """Aggregate per-video stage scores into corpus-level metrics.
63
```

```

64     `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
65     ``stages[stage]["score"]`` with tp/fp/fn/mean_iou/...). For each stage we
66     compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
67     a TP-weighted mean IoU, plus the macro mean of per-video F1.
68     """
69     out: Dict[str, dict] = {}
70     for stage in stages:
71         tp = fp = fn = 0
72         iou_weighted = 0.0
73         f1s: List[float] = []
74         n_videos = 0
75         for rep in per_video:
76             sr = rep.get("stages", {}).get(stage)
77             if not sr:
78                 continue
79             s = sr["score"]
80             tp += s["true_positives"]
81             fp += s["false_positives"]
82             fn += s["false_negatives"]
83             iou_weighted += s["mean_iou"] * s["true_positives"]
84             f1s.append(s["f1"])
85             n_videos += 1
86         precision = _safe_div(tp, tp + fp)
87         recall = _safe_div(tp, tp + fn)
88         out[stage] = {
89             "videos": n_videos,
90             "tp": tp, "fp": fp, "fn": fn,
91             "micro_precision": precision,
92             "micro_recall": recall,
93             "micro_f1": _safe_div(2 * precision * recall, precision + recall),
94             "macro_f1": _safe_div(sum(f1s), len(f1s)),
95             "mean_iou": _safe_div(iou_weighted, tp),
96         }
97     return out
98
99
100 def format_aggregate(agg: Dict[str, dict]) -> str:
101     """Human-readable corpus-level summary table."""
102     lines = [
103         "=" * 78,
104         " CORPUS AGGREGATE",
105         "=" * 78,
106         f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
107         f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
108         "-" * 78,
109     ]
110     for stage, a in agg.items():
111         lines.append(
112             f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
113             f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
114             f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
115         )
116     lines.append("=" * 78)
117     return "\n".join(lines)
118

```

10. user (2026-06-06T15:20:47.010Z)

```

1     """Score the pipeline's detected rallies against ground-truth annotations.
2
3     Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
4     so scoring is fast, deterministic, and free. Two prediction "stages" can be
5     scored:
6
7     - ``candidates`` ? raw detector windows (``candidate_segments`` table). What

```



```

73     for stage in stages:
74         preds = get_predictions(db, video_id, stage, detector=detector)
75         mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
76         s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
77         curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
78         report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
79                                           score=s, match_result=mr, pr_curve=curve)
80     return report
81
82
83     # ----- reporting
84
85     def _fmt_ts(seconds: float) -> str:
86         seconds = max(0, int(seconds))
87         h, rem = divmod(seconds, 3600)
88         m, s = divmod(rem, 60)
89         return f"{h:02d}:{m:02d}:{s:02d}"
90
91
92     def report_to_dict(report: EvalReport) -> dict:
93         """JSON-serialisable view of an EvalReport."""
94         out = {
95             "video_id": report.video_id,
96             "iou_threshold": report.iou_threshold,
97             "num_ground_truth": len(report.gt_intervals),
98             "stages": {},
99         }
100         for stage, sr in report.stages.items():
101             out["stages"][stage] = {
102                 "score": sr.score.as_dict(),
103                 "pr_curve": [s.as_dict() for s in sr.pr_curve],
104             }
105         return out
106
107
108     def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
109         """Human-readable console report for one video."""
110         lines = [
111             "=" * 64,
112             f" Rally evaluation ? video '{report.video_id}'",
113             f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold: {report.iou_threshold}",
114             "=" * 64,
115             f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
116             f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
117             "-" * 64,
118         ]
119         for stage in report.stages:
120             s = report.stages[stage].score
121             lines.append(
122                 f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4} "
123                 f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
124                 f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
125             )
126         lines.append("=" * 64)
127
128         if show_failures:
129             for stage, sr in report.stages.items():
130                 misses = [report.gt_intervals[i] for i in
131 sr.match_result.unmatched_gt_indices]
132                 fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
133                 lines.append(f"\n [{stage}] missed rallies (false negatives): {len(misses)}")
134                 for iv in misses:

```

```

134         lines.append(f"    - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
135         lines.append(f" [{stage}] spurious detections (false positives):
{len(fps)}")
136         for iv in fps:
137             lines.append(f"    - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
138
139         return "\n".join(lines)
140

```

11. user (2026-06-06T15:20:47.843Z)

```

1     """Build a training set for the offline, API-free learned rally segmenter.
2
3     Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
4     a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
5     produces candidate rally windows with features; the collector's ground-truth
6     rally intervals tell us which windows are real. A lightweight classifier learns
7     that mapping, replacing the paid Gemini refinement with a local model.
8
9     This module is the *data layer* ? pure and unit-tested:
10
11     - `label_candidates` ? generic: label windows positive/negative by whether they
12       match a ground-truth rally (same IoU matching the evaluator uses, so training
13       labels are consistent with how we score).
14     - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
15       persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
16     - `build_training_set` ? glue: rows + GT -> (X, y, intervals).
17
18     No model training or I/O here; that lives in the trainer/segmenter once the
19     substrate is chosen from the Phase 3 candidate-recall result.
20     """
21
22     from typing import Callable, Dict, List, Tuple
23
24     from backend.eval.metrics import Interval, match_intervals
25
26     # YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
27     # (see HybridYoloSegmenter). Duration is derived from the window bounds.
28     YOLO_FEATURE_NAMES = [
29         "duration", "peak_velocity", "mean_velocity", "active_frame_count",
30         "track_id_count",
31     ]
32
33     def extract_yolo_features(row: Dict) -> List[float]:
34         """Feature vector for one YOLO candidate row (dict from query_candidate_segments).
35
36         `stats` is already JSON-deserialized by the DB layer. Missing fields default
37         to 0.0 so a sparse/old row still yields a well-formed vector.
38         """
39         stats = row.get("stats") or {}
40         duration = float(row["end_time"]) - float(row["start_time"])
41         return [
42             duration,
43             float(stats.get("peak_velocity", 0.0)),
44             float(stats.get("mean_velocity", 0.0)),
45             float(stats.get("active_frame_count", 0)),
46             float(stats.get("track_id_count", 0)),
47         ]
48
49
50     def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
51                          iou_threshold: float = 0.5) -> List[int]:
52         """Label each candidate window 1 (real rally) or 0 (spurious).
53

```

```

54     A window is positive iff it is matched one-to-one to a ground-truth rally at
55     `iou_threshold` ? the *same* matching the evaluator uses for true positives,
56     so the classifier is trained on exactly what we score. Substrate-agnostic:
57     works for YOLO or WASB-derived windows alike.
58     """
59     mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
60     positive = {m.pred_index for m in mr.matches}
61     return [1 if i in positive else 0 for i in range(len(candidate_intervals))]
62
63
64 def build_training_set(
65     candidate_rows: List[Dict],
66     gt_intervals: List[Interval],
67     iou_threshold: float = 0.5,
68     feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
69 ) -> Tuple[List[List[float]], List[int], List[Interval]]:
70     """Turn one video's candidate rows + ground truth into (X, y, intervals).
71
72     `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
73     features without changing the labeling logic. Returns the feature matrix `X`,
74     binary labels `y`, and the candidate intervals (handy for downstream merging).
75     """
76     intervals: List[Interval] = [
77         (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
78     ]
79     y = label_candidates(intervals, gt_intervals, iou_threshold)
80     X = [feature_fn(r) for r in candidate_rows]
81     return X, y, intervals
82

```

12. user (2026-06-06T15:20:48.318Z)

```

1     """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3     The offline learned segmenter (ADR-004) must run locally with no external API and
4     no heavy ML stack ? the project already depends on numpy, and sklearn is not
5     installed. This implements just enough: standardized features, balanced-class
6     gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7     model is a small, human-readable, version-controllable file.
8
9     It deliberately stays simple: the rally-window features are few and roughly
10    linearly separable (high player motion + sustained activity => real rally), so a
11    well-regularized linear model is a sensible, inspectable baseline. Swap in a
12    richer model later behind the same fit/predict_proba/save/load interface.
13    """
14
15    import json
16    from typing import List, Optional
17
18    import numpy as np
19
20
21    class LogisticRegression:
22        def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23            self.lr = lr
24            self.epochs = epochs
25            self.l2 = l2
26            self.w: Optional[np.ndarray] = None
27            self.b: float = 0.0
28            self.mean: Optional[np.ndarray] = None
29            self.std: Optional[np.ndarray] = None
30            self.feature_names: Optional[List[str]] = None
31
32        @staticmethod
33        def _sigmoid(z: np.ndarray) -> np.ndarray:

```

```

34         # Clip for numerical stability (avoid overflow in exp).
35         return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
36
37     def _standardize(self, X: np.ndarray) -> np.ndarray:
38         return (X - self.mean) / self.std
39
40     def fit(self, X, y, feature_names: Optional[List[str]] = None,
41            class_weight: str = "balanced") -> "LogisticRegression":
42         """Fit with optional balanced class weighting (rally windows are imbalanced
43         ? many spurious candidates per real rally)."""
44         X = np.asarray(X, dtype=float)
45         y = np.asarray(y, dtype=float)
46         if X.ndim != 2 or X.shape[0] == 0:
47             raise ValueError("X must be a non-empty 2D array")
48         self.feature_names = feature_names
49
50         self.mean = X.mean(axis=0)
51         self.std = X.std(axis=0)
52         self.std[self.std == 0] = 1.0 # guard constant features
53         Xs = self._standardize(X)
54         n, d = Xs.shape
55
56         if class_weight == "balanced":
57             n_pos = max(1, int(y.sum()))
58             n_neg = max(1, int((1 - y).sum()))
59             w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
60         else:
61             w_pos = w_neg = 1.0
62         sw = np.where(y == 1, w_pos, w_neg)
63
64         self.w = np.zeros(d)
65         self.b = 0.0
66         for _ in range(self.epochs):
67             p = self._sigmoid(Xs @ self.w + self.b)
68             err = (p - y) * sw
69             grad_w = Xs.T @ err / n + self.l2 * self.w
70             grad_b = float(err.mean())
71             self.w -= self.lr * grad_w
72             self.b -= self.lr * grad_b
73         return self
74
75     def predict_proba(self, X) -> np.ndarray:
76         if self.w is None:
77             raise RuntimeError("model is not fitted")
78         Xs = self._standardize(np.asarray(X, dtype=float))
79         return self._sigmoid(Xs @ self.w + self.b)
80
81     def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82         return (self.predict_proba(X) >= threshold).astype(int)
83
84     # --- persistence (small JSON, no pickle/joblib) -----
85
86     def to_dict(self) -> dict:
87         return {
88             "model": "logreg",
89             "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90             "w": self.w.tolist() if self.w is not None else None,
91             "b": self.b,
92             "mean": self.mean.tolist() if self.mean is not None else None,
93             "std": self.std.tolist() if self.std is not None else None,
94             "feature_names": self.feature_names,
95         }
96
97     @classmethod
98     def from_dict(cls, d: dict) -> "LogisticRegression":

```

```

99         m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100         m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101         m.b = float(d.get("b", 0.0))
102         m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103         m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104         m.feature_names = d.get("feature_names")
105         return m
106
107     def save(self, path: str) -> None:
108         with open(path, "w") as f:
109             json.dump(self.to_dict(), f, indent=2)
110
111     @classmethod
112     def load(cls, path: str) -> "LogisticRegression":
113         with open(path) as f:
114             return cls.from_dict(json.load(f))
115

```

13. user (2026-06-06T15:20:48.554Z)

```

1     """Regression runner ? verify the production tool against golden-set ground truth.
2
3     Implements the explicit ask: "given a video, get a golden set created and verify
4     its precision and recall against the production tool."* This is deliberately a thin
5     wrapper over machinery that already exists:
6
7     ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8     predictions ? the production pipeline's stored DB rows [backend.eval.harness]
9     comparison ? interval matching + P/R/F1 [backend.eval.metrics]
10    baseline ? a per-video snapshot of "known good" numbers [this module]
11
12    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16    """
17    import os
18    import json
19    import logging
20    from dataclasses import dataclass, asdict
21    from typing import Dict, List, Optional
22
23    from backend.database import Database
24    from backend.eval import metrics
25    from backend.eval.harness import get_predictions
26    from backend.annotations import annotation_registry
27
28    logger = logging.getLogger(__name__)
29
30    DEFAULT_BASELINE_PATH = os.path.join("output", "regression_baseline.json")
31    DEFAULT_TOLERANCE = 0.05
32
33
34    @dataclass
35    class RegressionResult:
36        video_id: str
37        stage: str
38        iou_threshold: float
39        precision: float
40        recall: float
41        f1: float
42        # Baseline values compared against (None when no baseline existed yet).
43        baseline_precision: Optional[float]
44        baseline_recall: Optional[float]

```

```

45     tolerance: float
46     regressed: bool
47     reasons: List[str]
48
49     def as_dict(self) -> dict:
50         return asdict(self)
51
52
53 # ----- baseline store
54
55 def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
56     """Load the per-video baseline snapshot file (returns {} if absent)."""
57     if not os.path.isfile(path):
58         return {}
59     with open(path) as f:
60         return json.load(f)
61
62
63 def _baseline_key(video_id: str, stage: str) -> str:
64     return f"{video_id}::{stage}"
65
66
67 def save_baseline(video_id: str, stage: str, precision: float, recall: float,
68                 f1: float, path: str = DEFAULT_BASELINE_PATH) -> None:
69     """Snapshot a video/stage's current numbers as the new 'known good' baseline.
70
71     Use this to (re)baseline after a *legitimate* production improvement, so old
72     baselines don't raise false regressions. Stored as plain JSON for easy diffing.
73     """
74     baselines = load_baselines(path)
75     baselines[_baseline_key(video_id, stage)] = {
76         "video_id": video_id, "stage": stage,
77         "precision": precision, "recall": recall, "f1": f1,
78     }
79     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
80     with open(path, "w") as f:
81         json.dump(baselines, f, indent=2, sort_keys=True)
82     logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
83 _baseline_key(video_id, stage), precision, recall)
84
85 # ----- the runner
86
87 def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
88           stage: str = "final", iou_threshold: float = 0.5,
89           detector: Optional[str] = None,
90           tolerance: float = DEFAULT_TOLERANCE,
91           baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
92     """Score stored predictions for one video against ground truth and compare to
93 baseline.
94
95     `ground_truth` is a list of (start, end) intervals ? typically obtained from an
96     AnnotationProvider (see `regress_with_provider`). A regression is flagged when
97     precision OR recall falls more than `tolerance` below the snapshotted baseline.
98     If no baseline exists yet, `regressed` is False (nothing to compare to) and the
99     caller can choose to snapshot the current numbers via `save_baseline`.
100     """
101     preds = get_predictions(db, video_id, stage, detector=detector)
102     s = metrics.score(preds, ground_truth, iou_threshold)
103
104     baselines = load_baselines(baseline_path)
105     b = baselines.get(_baseline_key(video_id, stage))
106
107     reasons: List[str] = []
108     regressed = False

```

```

108     base_p = base_r = None
109     if b is not None:
110         base_p, base_r = b["precision"], b["recall"]
111         if s.precision < base_p - tolerance:
112             regressed = True
113             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
114         if s.recall < base_r - tolerance:
115             regressed = True
116             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
117     else:
118         reasons.append("no baseline recorded for this video/stage ? nothing to compare")
119
120     return RegressionResult(
121         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
122         precision=s.precision, recall=s.recall, f1=s.f1,
123         baseline_precision=base_p, baseline_recall=base_r,
124         tolerance=tolerance, regressed=regressed, reasons=reasons,
125     )
126
127
128     def regress_with_provider(db: Database, video_id: str, video_ref: str,
129                             tier: str = "file", stage: str = "final",
130                             iou_threshold: float = 0.5,
131                             detector: Optional[str] = None,
132                             tolerance: float = DEFAULT_TOLERANCE,
133                             baseline_path: str = DEFAULT_BASELINE_PATH,
134                             guideline: Optional[str] = None,
135                             sport: str = "badminton",
136                             provider_kwargs: Optional[dict] = None) -> RegressionResult:
137         """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
138
139         `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
140         once those providers land). `video_ref` is whatever that provider expects (a CSV
141         path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
142         predictions in the DB (the file MD5).
143         """
144         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
145         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
146         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
147                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)
148
149
150     def format_result(r: RegressionResult) -> str:
151         verdict = "REGRESSED" if r.regressed else "ok"
152         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
153               f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
154         if r.baseline_precision is not None:
155             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
156         return head + ".join(f'\n    - {why}' for why in r.reasons)
157

```

14. assistant (2026-06-06T15:20:59.479Z)

I have all files read. Let me verify a couple of cross-module references to make sure the edges are accurate ? the `annotation\_registry` and `TrackNetRunner` methods consumed by these files.

15. user (2026-06-06T15:21:01.844Z)

```

204:     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
223:     def trajectory_to_action_windows(

```

16. user (2026-06-06T15:21:02.503Z)

```
backend\annotations\file_provider.py:16:    annotation_registry,
backend\annotations\file_provider.py:24:class FileProvider(AnnotationProvider):
backend\annotations\file_provider.py:66:annotation_registry.register(FileProvider)
backend\annotations\automated_provider.py:24:    annotation_registry,
backend\annotations\automated_provider.py:45:        "annotation_registry.get('auto',
labeler=<fn>, lineage='<model-family>'). "
backend\annotations\automated_provider.py:66:class AutomatedProvider(AnnotationProvider):
backend\annotations\automated_provider.py:103:annotation_registry.register(AutomatedProvider)
backend\annotations\base.py:28:class AnnotationProvider(ABC):
backend\annotations\base.py:63:    def annotate(self, video_ref: str, guideline: Optional[str] =
None,
backend\annotations\base.py:81:class AnnotationProviderRegistry:
backend\annotations\base.py:110:annotation_registry = AnnotationProviderRegistry()
backend\annotations\__init__.py:7:from .base import annotation_registry, AnnotationProvider
backend\annotations\__init__.py:11:__all__ = ["annotation_registry", "AnnotationProvider"]
```

17. assistant (2026-06-06T15:21:04.858Z)

Cross-references confirmed. I have everything needed to build the code map.

18. user (2026-06-06T15:23:17.747Z)

Structured output provided successfully

19. assistant (2026-06-06T15:23:31.578Z)

Code map for the Eval & calibration subsystem has been returned via the structured output tool.

Key takeaways for a future engineer ramping up:

- `backend/eval/metrics.py` is the foundation ? temporal-IoU greedy one-to-one interval matching producing `Score` (P/R/F1/mean-IoU/boundary errors). Everything else delegates to it.
- Two parallel layers sit on top: the **DB-scoring** path (`harness.py` reads stored predictions ? `batch.py` aggregates corpus-wide ? `regression.py` guards against drops vs a JSON baseline) and the **calibration** path (`calibration.py` pure overfit-guards parameterised by a `predict\_fn(config)`, driven by the WASB CLIs `calibrate\_wasb.py` / `export\_wasb\_segments.py`).
- The **learned-segmenter** path (Phase 4): `training.py` builds `(X, y, intervals)` using the **same** IoU matching the evaluator scores with, feeding `classifier.py`'s numpy-only logistic regression.

Two footguns worth flagging up front:

- There are **two different "annotations" modules**: `backend/eval/annotations.py` (generic `start,end` GT CSV, fails loud) vs `backend/annotations` (the provider registry that `regression.py` imports). They also use different CSV schemas (`start,end` vs `start\_time,end\_time`), and the two GT loaders have opposite failure philosophies (annotations.py raises on bad rows; `calibrate\_wasb.load\_golden\_gts` silently skips them).
- `calibrate\_wasb.run` selects the grid on the full GT, so its `improvement\_report` "+X%" is an optimistic upper bound ? the honest number is the cross-validated held-out recall (and `leave\_one\_video\_out` once a 2nd labelled video exists). Both **BASELINE** (calibrate_wasb) and **TUNED** (export_wasb_segments) are hardcoded constants that must be kept in sync with the real WASB config / latest calibration.